BATCH 26

Inceptez Data Science Presents

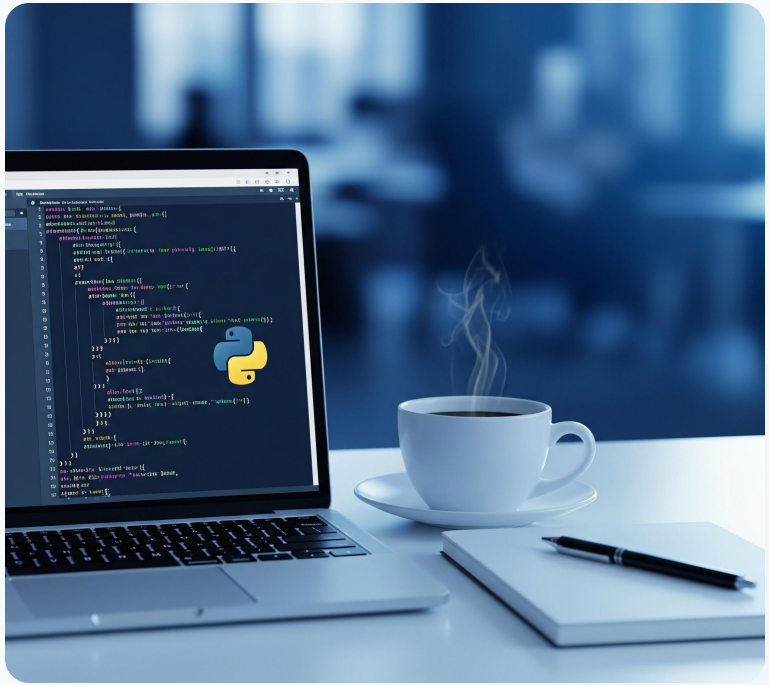
# Python Day - 2

Laxmi Narayen Nagarajan Venkatesan

19th-April-2026

REVIEW

# Reflecting on Our Last Sessions



Anaconda Navigator

ANACONDA.NAVIGATOR

Connect

Home

Environments

Learning

Community

All applications on base (root) Channels

**PyCharm Professional**
A full-fledged IDE by JetBrains for both Scientific and Web Python development. Supports HTML, JS, and SQL.
Install

**Anaconda Toolbox**
0.4.0
Anaconda Assistant
JupyterLab supercharged with a suite of Anaconda extensions, starting with the Anaconda Assistant AI chatbot.
Install

**Anaconda Cloud Notebooks**
Cloud-hosted notebook service from Anaconda. Launch a preconfigured environment with hundreds of packages and store project files with persistent cloud storage.
Launch

**Notebook**
7.0.6
Web-based, interactive computing notebook environment. Edit and run human-readable docs while describing the data analysis.
Launch

**Qt Console**
5.4.2
PyQt GUI that supports inline figures, proper multiline editing with syntax highlighting, graphical calltips, and more.
Launch

**Spyder**
5.4.3
Scientific PYthon Development EnviRonment. Powerful Python IDE with advanced editing, interactive testing, debugging and introspection features
Launch

**VS Code**
1.87.2
Streamlined code editor with support for development operations like debugging, task running and version control.
Launch

**Anaconda on AWS Graviton**
Running your Anaconda workloads on AWS Graviton-based processors could provide up to 40% better price performance
Launch

**Oracle Data Science Service**
OCI Data Science offers a machine learning platform to build, train, manage, and deploy your machine learning models on the cloud with your favorite open-source tools
Launch

**Glueviz**
1.2.4
Multidimensional data visualization across files. Explore relationships within and among related datasets.
Install

**JupyterLab**
4.0.11
An extensible environment for interactive and reproducible computing, based on the Jupyter Notebook and Architecture.
Install

**Orange 3**
3.34.0
Component based data mining framework. Data visualization and data analysis for novice and expert. Interactive workflows with a large toolbox.
Install

NEW
Anaconda Toolbox
Supercharged local notebooks. Click the Toolbox tile to install.
Read the Docs

Documentation

Anaconda Blog

Review

# Ways to Run Python in 2026

## CLI - codes

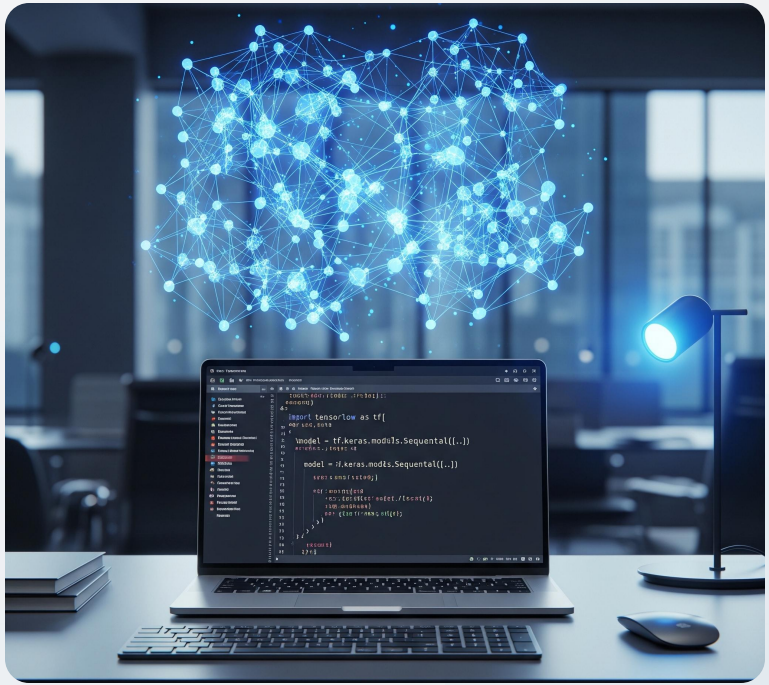
**Terminal:** Python - Python Terminal

## Interactive Notebooks - local ide / VSCode

**Jupyter & Colab:** Leveraging kernels for rapid prototyping, rich visualization, and collaborative cloud computing.

## Python scripts

Either use partitioned virtual Envs / Base envs to write python scripts



1. CLI - Terminal - type python

>>> <type your python code>

2. Jupyter and star writing python codes in cells inside notebook - ipynb

Interactive python notebook files - Any cells can be run any time / order

3. Running as .py files - Create new txt.. Rename it to <name>.py

Run it using command: **python <name>.py**

4. Reusing previous codes - Put the reusable part of the code including values of variables inside a .py file - get it inside jupyter or another .py as well via **“import”** keyword

Review

# Data Types — The Five Basic Flavors

## 1. Integer (int)

**"Count Anything"**

Whole numbers, exact quantities.

Ex: 11,000 students, 10 months.

## 2. Float (float)

**"Measure Anything"**

Precision numbers with decimals.

Ex: 4.8 rating, 85.0% hands-on.

## 3. Boolean (bool)

**"Yes or No"**

Simplest type for logic decisions.

Values: True / False

## 4. String (str)

**"Letters & Text"**

Wrapped in quotes to identify as text.

Ex: "inceptez", "2026"

## 5. Complex

**"AI Research"**

Advanced math with imaginary parts.

Ex: 3 + 5j

## 💡 Analogy: Ingredients vs. Recipe

Think of data types as categories of ingredients. You wouldn't use a "string" (text) to perform "math" (multiplication) any more than you'd use a spoon to cut an onion. Handling data correctly requires knowing its **flavor**.

## 👩‍🏫 Lab Activity

What's My Type?

```
1. "11/04/2026"
2. 12
3. 85.0
4. True
```

*Instructor Ans: 1.Str, 2.Int, 3.Float, 4.Bool*

CONCEPT

# Data Type Casting — The Art of Conversion

## Implicit Casting (Automatic)

Python automatically converts one data type to another without user involvement to avoid data loss.

```
x = 10 + 5.5 # Result: 15.5 (float)
```

## Explicit Casting (Manual)

User manually converts data types using built-in constructor functions:

- **int()**: Constructs an integer from a float or string.
- **float()**: Constructs a float from an integer or string.
- **str()**: Constructs a string from any type.

Example:

```
age = int("25") # String to Int
price = float(10) # Int to Float
```

💡 Tip: Python cannot cast non-numeric strings to integers (e.g., int("Hello") results in a ValueError).

Review

# The Data Structure "DNA" Comparison

### 🟦 List

**Type:** `[ ]`

**Heterogeneity:** **Supported**

**Index:** **Supported (0)**

**Mutability:** **Mutable via index**
**Mutability means ability to change the values of the data structure**

### 🔒 Tuple

**Type:** `( )`

**Heterogeneity:** **Supported**

**Index:** **Supported**

**Mutability:** **Immutable**

### 📖 Dictionary

**Type:** `{ k:v }`

**Heterogeneity:** **Supported**

**Index:** Key-based

**Mutability:** **Mutable**

### 🧪 Set

**Type:** `{ }`

**Heterogeneity:** **Supported**

**Index:** No Position

**Mutability:** **No change via index**

### The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 1

# The Data Structure "DNA" Comparison

### 🟦 List

**Type:** [ ]

**Heterogeneity:** Supported

**Index:** Supported (0)

**Mutability:** Mutable

### 🔒 Tuple

**Type:** ( )

**Heterogeneity:** Supported

**Index:** Supported

**Mutability:** Immutable

### 📖 Dictionary

**Type:** { k:v }

**Heterogeneity:** Supported

**Index:** Key-based

**Mutability:** Mutable

### 🧪 Set

**Type:** { }

**Heterogeneity:** Supported

**Index:** No Position

**Mutability:** Mutable

### The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 1

# Python List Operations

## 1. Basic Operations

- Access: `list[index]`
- Update: `list[i] = val`
- Delete: `del list[i]`
- Length: `len(list)`

## 2. Core Operations

- Add: `.append()`, `.insert()`
- Combine: `.extend()`
- Remove: `.pop()`, `.remove()`
- Sort: `.sort()`, `.reverse()`

## 3. Special Operations

- Slicing: `list[start:stop:step]`
- Membership: `val in list`
- Repetition: `list * 3`

CONCEPT

# Python Slicing & Negative Indexing

## Advanced Slicing: [start:stop:step]

Slicing creates a new list from a range of indices.

**Syntax:** `list[start : stop : step]`

- **Stop** is exclusive (up to, but not including).
- **Step** determines the increment (default is 1).

## Negative Indexing: Backward Access

Access elements from the end of the list without knowing its length.

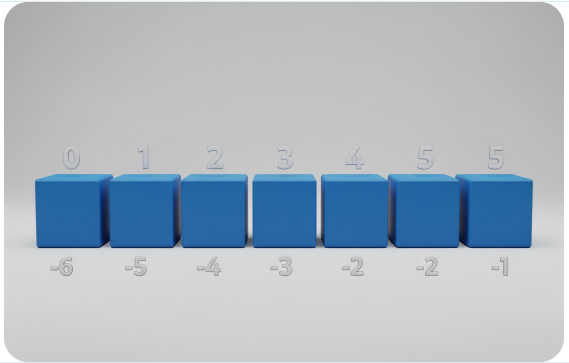
**Last Element:** `list[-1]`

- **-1** is the last item, **-2** is second-to-last.
- Highly useful for dynamic data processing.

## Visualizing the Indices

**Example List:** `nums = [10, 20, 30, 40, 50]`

- **Forward Slice:** `nums[1:4]` → `[20, 30, 40]`
- **Negative Slice:** `nums[-3:]` → `[30, 40, 50]`
- **Reverse List:** `nums[::-1]` → `[50, 40, 30, 20, 10]`



PRACTICE

# List Challenge: Solve & Code

## List Mirroring

Reverse the input list and append it to the original list to create a "mirror" effect.

**Sample Input:**
`["A", "B"]`

**Expected Output:**
`["A", "B", "B", "A"]`

*Hint: Use .reverse() or slicing [::-1] then add (+).*

SOLUTION

# Challenge Solution: Pythonic Lists

## 2. List Mirroring

Slicing provides a non-destructive way to reverse and concatenate lists.

**Python Solution:**

```
data = ["A", "B"]
mirror = data + data[::-1]
print(mirror) # ["A", "B", "B", "A"]
```

PART 2

# The Data Structure "DNA" Comparison

### 🟦 List

**Type:** [ ]

**Heterogeneity:** Supported

**Index:** Supported (0)

**Mutability:** Mutable

### 🔒 Tuple

**Type:** ( )

**Heterogeneity:** Supported

**Index:** Supported

**Mutability:** Immutable

### 📖 Dictionary

**Type:** { k:v }

**Heterogeneity:** Supported

**Index:** Key-based

**Mutability:** Mutable

### 🧪 Set

**Type:** { }

**Heterogeneity:** Supported

**Index:** No Position

**Mutability:** Mutable

### The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

Part 2

# Python Tuple Operations

## Indexing & Slicing

Identical to lists, access data via position or range.

**Access:** `tup[0]` or `tup[-1]`
**Slice:** `tup[1:3]`

## Combination & Repetition

Join tuples or repeat elements to create new tuples.

**Concatenate:** `t1 + t2`
**Multiply:** `t1 * 3`

## Built-in Tuple Methods

Tuples only have two methods due to their immutability.

- **count(x):** Returns number of times x appears.
- **index(x):** Finds the first index of value x.

💡 **Pro Tip:** Since tuples are immutable, "operations" like addition actually return a brand new tuple object.

PART 3

# The Data Structure "DNA" Comparison

### 🟦 List

**Type:** `[ ]`

**Heterogeneity:** Supported

**Index:** Supported (0)

**Mutability:** Mutable

### 🔒 Tuple

**Type:** `( )`

**Heterogeneity:** Supported

**Index:** Supported

**Mutability:** Immutable

### 📖 Dictionary

**Type:** `{ k:v }`

**Heterogeneity:** Supported

**Index:** Key-based

**Mutability:** Mutable

### 🧪 Set

**Type:** `{ }`

**Heterogeneity:** Supported

**Index:** No Position

**Mutability:** Mutable

### The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 3

# Python Set Operations

## Mathematical Operations

Sets excel at unique membership and comparisons.

**Union:** `s1 | s2` or `s1.union(s2)`

**Intersection:** `s1 & s2`

## Difference & Symmetric Diff

Identify elements unique to specific sets.

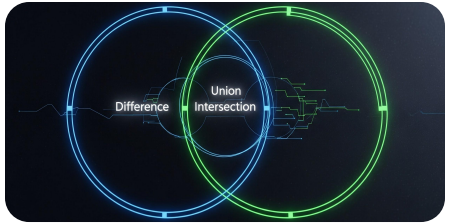
**Difference:** `s1 - s2`

**Symmetric:** `s1 ^ s2`

## Common Set Methods

Unlike tuples, sets are mutable and support modifications.

- **add(x):** Adds an element to the set.
- **remove(x):** Removes x; raises error if not found.




💡 **Pro Tip:** Sets are unordered; you cannot access elements via index like lists or tuples.

PART 4

# The Data Structure "DNA" Comparison

### 🟦 List

**Type:** `[ ]`

**Heterogeneity:** **Supported**

**Index:** **Supported (0)**

**Mutability:** **Mutable**

### 🔒 Tuple

**Type:** `( )`

**Heterogeneity:** **Supported**

**Index:** **Supported**

**Mutability:** **Immutable**

### 📖 Dictionary

**Type:** `{ k:v }`

**Heterogeneity:** **Supported**

**Index:** Key-based

**Mutability:** **Mutable**

### 🧪 Set

**Type:** `{ }`

**Heterogeneity:** **Supported**

**Index:** No Position

**Mutability:** **Mutable**

### The "Four-Point Check" Strategy

1. Type (Brackets), 2. Heterogeneity (Mixed data?), 3. Index (Position number?), 4. Mutability (Can it change?).

PART 4

# Python Dictionary Operations

## Key-Value Management

Access, update, or add pairs using unique keys.

**Access:** `dict["key"]` or `.get()`
**Update/Add:** `dict["k"] = "v"`

## Dictionary View Objects

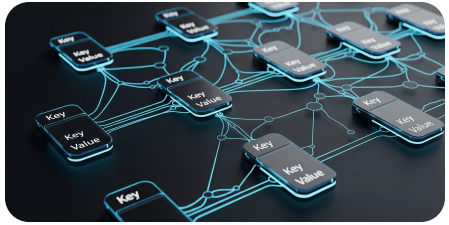
Dynamically view keys, values, or items.

**Keys:** `.keys()`
**Values:** `.values()`
**Pairs:** `.items()`

## Core Methods & Cleanup

Dictionaries provide robust tools for safe data removal.

- **pop(key):** Removes key and returns its value.
- **update(dict2):** Merges two dictionaries together.




💡 **Pro Tip:** Since Python 3.7+, dictionaries maintain insertion order for all keys.

PART 5

# Data Structure Type Casting

## Standard Casting Functions

Convert between sequences using constructor functions.

**To List:** `list(iterable)`

**To Tuple:** `tuple(iterable)`

**To Set:** `set(iterable)`

## Special Case: Dictionaries

Requires paired data (key-value) to cast successfully.

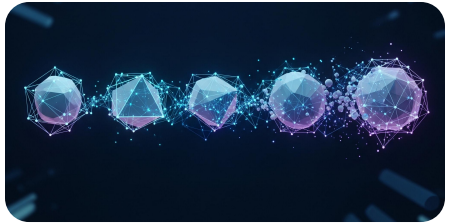
**From Pairs:** `dict([('a', 1), ('b', 2)])`

**Zip:** `dict(zip(keys, values))`

## Common Conversion Scenarios

Data transformation is essential for optimizing performance or ensuring data safety.

- **Deduplication:** Cast a list to a `set()` to remove duplicates.
- **Freezing:** Cast a list to a `tuple()` to make it immutable.
- **Sorting:** Cast sets/dicts to a `list()` to apply custom sorts.



💡 **Pro Tip:** Casting from a Dictionary directly to a List/Set only retrieves the `keys` by default.

PART 6

# Control Flow: Loops & Conditionals

## Conditional Control

Logic gates that execute code based on specific truths.

**Structure:** `if`, `elif`, `else`

**Usage:** Branches the program path based on Boolean results.

## Iterative Loops

Repeating a block of code multiple times until a condition is met.

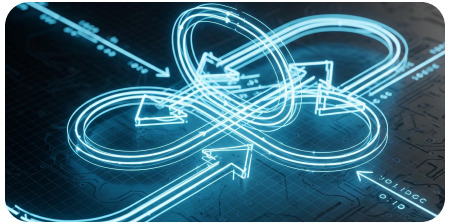
**Keyword:** `for` (sequence) or `while` (condition)

**Goal:** Automate repetitive tasks efficiently across data sets.

## Looping Strategies

Choosing the right tool for the job ensures clean, readable code.

- **For Loops:** Ideal for iterating over lists, strings, or ranges.
- **While Loops:** Used when the number of iterations is unknown.



💡 **Pro Tip:** Always ensure a "break" condition in `while` loops to avoid infinite execution.

PART 6

# Optimizing Looping Strategies

## For Loops: Definite Iteration

Ideal for iterating over known sequences like `lists`, `strings`, or `ranges`. Use when the number of steps is pre-determined.

## While Loops: Indefinite Iteration

Used when the number of iterations is unknown. Executes as long as a condition remains `True`. Perfect for event listening or state-based logic.

💡 **Pro Tip:** Choosing the right tool for the job ensures clean, readable, and performant code.

MOTIVATION

# The Power of Consistency

## Build Your Learning Streak

Success in programming isn't about one-time brilliance; it's about the **habit of showing up**. Every class attended adds to your momentum.

## Invest in Your Future Self

As Warren Buffett says, the best investment is **in yourself**. Small, daily gains lead to an unfathomable difference in career outcomes over time.

💡 **Pro Tip:** Consistent effort is the slight edge that separates average coders from world-class engineers.